

# Alur Logika Endpoint — Spring Boot Implementation

Struktur ini pakai layered architecture standar Spring

**Boot:** Controller → Service (orchestrator) → Service-service kecil per tanggung jawab. **Tiap class fokus satu hal (single responsibility)** biar gampang ditest.

---

## 1. Struktur Package

```
com.atscvreview
├── controller
│   └── CvAnalysisController.java
├── service
│   ├── CvAnalysisOrchestrator.java    //
│   │   orkestrasi semua step
│   ├── RateLimitService.java
│   ├── FileValidationService.java
│   ├── TextExtractionService.java
│   ├── ParsingGateService.java       // Gate 1
│   └── AiAnalysisService.java        // Gate 2 &
│   3 (panggil LLM)
│       ├── ScoringService.java
│       └── PdfReportService.java
├── dto
│   ├── AnalysisRequest.java
│   ├── AnalysisResponse.java
│   ├── ParsingResult.java
│   ├── AiAnalysisResult.java
│   └── StructuralInfo.java
```

```
├─ entity
│   └─ AnalysisResult.java
├─ repository
│   └─ AnalysisResultRepository.java
├─ config
│   └─ ScoringRulesConfig.java      // load
rules dari JSON/DB
└─ exception
    └─ GlobalExceptionHandler.java
```

---

## 2. DTO — Request & Response

```
// dto/AnalysisRequest.java
public record AnalysisRequest(
    MultipartFile cvFile,
    String jobDescription
) {}

// dto/StructuralInfo.java
public record StructuralInfo(
    boolean isMultiColumn,
    boolean hasTable,
    boolean hasTextBox,
    boolean hasImage,
    List<String> fontsUsed,
    List<String> detectedHeadings,
    boolean contactInfoInHeaderFooter,
    int extractedTextLength
) {}

// dto/ParsingResult.java
public record ParsingResult(
    int parsingScore,
    List<Issue> issues
) {}
```

```
// dto/AiAnalysisResult.java
public record AiAnalysisResult(
    int keywordScore,
    int completenessScore,
    List<String> matchedKeywords,
    List<String> missingKeywords,
    List<Issue> keywordIssues,
    List<RewriteSuggestion> rewriteSuggestions
) {}

public record Issue(String category, String
description, String severity) {}
public record RewriteSuggestion(String original,
String suggestion, String reason) {}

// dto/AnalysisResponse.java
public record AnalysisResponse(
    int totalScore,
    int parsingScore,
    int keywordScore,
    int completenessScore,
    List<Issue> issues,
    List<RewriteSuggestion> rewriteSuggestions,
    String pdfDownloadUrl
) {}
```

---

### 3. Controller — Entry Point

```
@RestController
@RequestMapping("/api/cv")
public class CvAnalysisController {

    private final CvAnalysisOrchestrator
orchestrator;

    public
```

```

CvAnalysisController(CvAnalysisOrchestrator
orchestrator) {
    this.orchestrator = orchestrator;
}

@PostMapping(value = "/analyze", consumes =
MediaType.MULTIPART_FORM_DATA_VALUE)
public ResponseEntity<AnalysisResponse>
analyze(
    @RequestParam("file") MultipartFile
file,
    @RequestParam("jd") String
jobDescription,
    HttpServletRequest request
) {
    String clientKey =
resolveClientKey(request); // IP atau userId dari
JWT

    AnalysisResponse result =
orchestrator.process(file, jobDescription,
clientKey);
    return ResponseEntity.ok(result);
}

private String
resolveClientKey(HttpServletRequest request) {
    // kalau ada JWT/auth principal, pakai
userId. Kalau guest, pakai IP.
    return request.getRemoteAddr();
}
}

```

**Catatan: validasi error (400/422/429) dilempar sebagai custom exception dari service layer, ditangkap `GlobalExceptionHandler` — controller tetap bersih, gak ada if-else validasi di sini.**

---

## 4. Orchestrator — Jantung Alur Logika

Ini class yang manggil semua step berurutan, persis sesuai alur yang udah dirancang sebelumnya.

```
@Service
public class CvAnalysisOrchestrator {

    private final RateLimitService
    rateLimitService;
    private final FileValidationService
    fileValidationService;
    private final TextExtractionService
    textExtractionService;
    private final ParsingGateService
    parsingGateService;
    private final AiAnalysisService
    aiAnalysisService;
    private final ScoringService scoringService;
    private final PdfReportService
    pdfReportService;
    private final AnalysisResultRepository
    repository;

    // constructor injection semua dependency di
    atas...

    public AnalysisResponse process(MultipartFile
    file, String jd, String clientKey) {

        // Step 0 – Rate limit check
        rateLimitService.checkLimit(clientKey); //
        throws RateLimitExceededException kalau habis

        // Step 1 – Validasi input
        fileValidationService.validate(file, jd);
        // throws InvalidInputException kalau gagal

        // Step 2 – Extract text + info struktural
```

```

        ExtractedContent extracted =
textExtractionService.extract(file);
        // extracted berisi: rawText +
StructuralInfo

        // Step 3 – Gate 1: Parsing check (rule-
based, TANPA AI)
        ParsingResult parsingResult =
parsingGateService.evaluate(extracted.structuralInf
o(), extracted.rawText());

        // Step 4 & 5 – Gate 2 & 3: AI call
(keyword matching + completeness + rewrite
suggestion)
        AiAnalysisResult aiResult =
aiAnalysisService.analyze(extracted.rawText(), jd);

        // Step 6 – Hitung skor final (weighted
formula dari config)
        int totalScore =
scoringService.calculateTotal(
            parsingResult.parsingScore(),
            aiResult.keywordScore(),
            aiResult.completenessScore()
        );

        List<Issue> allIssues =
mergeIssues(parsingResult.issues(),
aiResult.keywordIssues());

        // Step 7 – Generate PDF report
        String pdfUrl =
pdfReportService.generate(totalScore,
parsingResult, aiResult, allIssues);

        // Step 8 – Simpan hasil kalau user login
(optional, cek clientKey punya userId atau bukan)
        if (isAuthenticatedUser(clientKey)) {
            repository.save(buildEntity(totalScore,
parsingResult, aiResult, pdfUrl));

```

```

    }

    // Step 9 – Return response
    return new AnalysisResponse(
        totalScore,
        parsingResult.parsingScore(),
        aiResult.keywordScore(),
        aiResult.completenessScore(),
        allIssues,
        aiResult.rewriteSuggestions(),
        pdfUrl
    );
}
}

```

## 5. RateLimitService — Bucket4j

```

@Service
public class RateLimitService {

    private final Map<String, Bucket> cache = new
    ConcurrentHashMap<>();

    public void checkLimit(String clientKey) {
        Bucket bucket =
        cache.computeIfAbsent(clientKey, k ->
        createNewBucket());
        if (!bucket.tryConsume(1)) {
            throw new
            RateLimitExceededException("Batas analisa harian
            tercapai");
        }
    }

    private Bucket createNewBucket() {
        Bandwidth limit = Bandwidth.classic(3,

```

```

Refill.intervally(3, Duration.ofDays(1));
    return
    Bucket.builder().addLimit(limit).build();
    }
}

```

---

## 6. FileValidationService

```

@Service
public class FileValidationService {

    private static final Set<String> ALLOWED_TYPES
= Set.of(
        "application/pdf",
        "application/vnd.openxmlformats-
officedocument.wordprocessingml.document"
    );
    private static final long MAX_SIZE = 5 * 1024 *
1024; // 5MB

    public void validate(MultipartFile file, String
jd) {
        if (file == null || file.isEmpty()) {
            throw new InvalidInputException("File
CV wajib diupload");
        }
        if
(!ALLOWED_TYPES.contains(file.getContentType())) {
            throw new InvalidInputException("Format
file tidak didukung, gunakan PDF atau DOCX");
        }
        if (file.getSize() > MAX_SIZE) {
            throw new InvalidInputException("Ukuran
file maksimal 5MB");
        }
        if (jd == null || jd.trim().length() < 50)

```

```

{
    throw new InvalidInputException("Job
description terlalu pendek atau kosong");
}
}
}

```

---

## 7. TextExtractionService — PDFBox / POI

```

@Service
public class TextExtractionService {

    public ExtractedContent extract(MultipartFile
file) {
        String contentType = file.getContentType();
        try {
            if
("application/pdf".equals(contentType)) {
                return extractFromPdf(file);
            } else {
                return extractFromDocx(file);
            }
        } catch (IOException e) {
            throw new
TextExtractionException("Gagal membaca isi CV", e);
        }
    }

    private ExtractedContent
extractFromPdf(MultipartFile file) throws
IOException {
        try (PDDocument document =
Loader.loadPDF(file.getBytes())) {
            PDFTextStripper stripper = new

```

```

PDFTextStripper();
    String rawText =
stripper.getText(document);

    StructuralInfo structuralInfo =
analyzePdfStructure(document); // deteksi kolom,
font, dsb
    return new ExtractedContent(rawText,
structuralInfo);
    }
}

private ExtractedContent
extractFromDocx(MultipartFile file) throws
IOException {
    try (XWPFDocument document = new
XWPFDocument(file.getInputStream())) {
        XWPFFWordExtractor extractor = new
XWPFFWordExtractor(document);
        String rawText = extractor.getText();

        StructuralInfo structuralInfo =
analyzeDocxStructure(document);
        return new ExtractedContent(rawText,
structuralInfo);
    }
}

private StructuralInfo
analyzePdfStructure(PDDocument document) {
    // gunakan PDFTextStripper custom (override
processTextPosition) untuk:
    // - deteksi multi-column dari sebaran
koordinat X
    // - kumpulkan font name per run teks
    // - deteksi tabel dari alignment grid
(heuristik posisi)
    // implementasi detail di luar scope contoh
ini
    ...

```

```

    }

    private StructuralInfo
    analyzeDocxStructure(XWPFDocument document) {
        // POI expose getTables(), paragraph run
        font info, dsb
        ...
    }
}

public record ExtractedContent(String rawText,
    StructuralInfo structuralInfo) {}

```

---

## 8. ParsingGateService — Gate 1 (Rule Engine)

```

@Service
public class ParsingGateService {

    private final ScoringRulesConfig config;

    public ParsingGateService(ScoringRulesConfig
    config) {
        this.config = config;
    }

    public ParsingResult evaluate(StructuralInfo
    info, String rawText) {
        List<Issue> issues = new ArrayList<>();

        if (info.isMultiColumn()) {
            issues.add(new Issue("layout", "CV
            menggunakan multi-column, ATS sering gagal parsing
            ini", "high"));
        }
    }
}

```

```

        if (info.hasTable()) {
            issues.add(new Issue("layout",
                "Terdapat tabel yang berisiko gagal terbaca ATS",
                "high"));
        }
        if (info.hasTextBox()) {
            issues.add(new Issue("layout",
                "Terdapat text box, konten di dalamnya sering tidak
                terbaca ATS", "high"));
        }
        if (info.hasImage()) {
            issues.add(new Issue("layout",
                "Terdapat gambar/ikon/skill bar, ATS tidak bisa
                ekstrak teks dari visual", "medium"));
        }
        if
        (!config.getFontWhitelist().containsAll(info.fontsUsed())) {
            issues.add(new Issue("font", "Font yang
                dipakai bukan font standar ATS-safe", "medium"));
        }
        List<String> missingHeadings =
            config.getHeadingWhitelist().stream()
                .filter(h ->
                    !info.detectedHeadings().contains(h))
                .toList();
        if (!missingHeadings.isEmpty()) {
            issues.add(new Issue("structure",
                "Section heading tidak standar: " +
                missingHeadings, "medium"));
        }
        if (info.contactInfoInHeaderFooter()) {
            issues.add(new Issue("contact", "Info
                kontak ada di header/footer, berisiko hilang saat
                parsing", "high"));
        }
        if (info.extractedTextLength() <
            config.getMinTextLength()) {
            issues.add(new Issue("integrity", "Teks
                hasil ekstraksi terlalu pendek, kemungkinan CV

```

```

berbasis gambar/scan", "high"));
    }

    int score = calculateParsingScore(issues);
    return new ParsingResult(score, issues);
}

private int calculateParsingScore(List<Issue>
issues) {
    int score = 100;
    for (Issue issue : issues) {
        score -= switch (issue.severity()) {
            case "high" -> 20;
            case "medium" -> 10;
            default -> 5;
        };
    }
    return Math.max(score, 0);
}
}

```

---

## 9. AiAnalysisService — Gate 2 & 3 (Panggil LLM)

```

@Service
public class AiAnalysisService {

    private final WebClient webClient; //
    konfigurasi base-url + API key di WebClient bean
    private final ScoringRulesConfig config;
    private final ObjectMapper objectMapper;

    public AiAnalysisResult analyze(String cvText,
String jobDescription) {
        String systemPrompt = buildSystemPrompt();

```

```

        String userMessage =
buildUserMessage(cvText, jobDescription);

        String rawResponse = callLlm(systemPrompt,
userMessage);
        return parseAndValidate(rawResponse, /*
retry callback */ () -> callLlm(systemPrompt,
userMessage));
    }

    private String buildSystemPrompt() {
        return ""
            Kamu adalah sistem analisa CV terhadap
job description, meniru cara kerja ATS.
            Rules yang WAJIB diikuti:
                1. Matching keyword bersifat literal,
jangan asumsikan sinonim otomatis.
                2. Nilai posisi keyword: keyword di
bullet pencapaian lebih kredibel daripada di skill
list saja.
                3. Density optimal keyword: %d-%d
repetisi, di atas itu anggap stuffing.
                4. Untuk bullet yang cuma deskripsi
tanggung jawab (bukan pencapaian terukur),
                    berikan rewrite pakai formula:
Action verb + keyword + hasil terukur.
                5. JANGAN mengarang angka/statistik
yang tidak ada di CV asli.
                    Kalau data kuantitatif tidak
tersedia, berikan placeholder eksplisit
                    contoh "[isi: berapa % peningkatan]"
untuk diisi user sendiri.
                6. WAJIB balas HANYA dalam format JSON
sesuai schema berikut, tanpa teks lain:
                    { "keywordScore": int,
"completenessScore": int, "matchedKeywords": [...],
"missingKeywords": [...],
"keywordIssues": [...], "rewriteSuggestions": [...]
                    }
    }

```

```

"".formatted(config.getKeywordDensityMin(),
config.getKeywordDensityMax());
    }

    private String buildUserMessage(String cvText,
String jd) {
        return "CV:\n" + cvText + "\n\nJob
Description:\n" + jd;
    }

    private String callLlm(String systemPrompt,
String userMessage) {
        // panggil endpoint LLM via WebClient,
model murah (mis. Haiku) untuk MVP
        // return raw text response (diharapkan
JSON string)
        ...
    }

    private AiAnalysisResult
parseAndValidate(String rawResponse,
Supplier<String> retryFn) {
        try {
            return
objectMapper.readValue(rawResponse,
AiAnalysisResult.class);
        } catch (JsonProcessingException e) {
            // retry sekali kalau format gagal
            String retried = retryFn.get();
            try {
                return
objectMapper.readValue(retried,
AiAnalysisResult.class);
            } catch (JsonProcessingException e2) {
                throw new
AiResponseParsingException("Gagal parsing response
AI setelah retry", e2);
            }
        }
    }
}

```

```
    }  
}
```

---

## 10. ScoringService — Weighted Formula

```
@Service  
public class ScoringService {  
  
    private final ScoringRulesConfig config;  
  
    public int calculateTotal(int parsingScore, int  
keywordScore, int completenessScore) {  
        double total = (parsingScore *  
config.getWeightParsing()  
                        + (keywordScore *  
config.getWeightKeyword()  
                        + (completenessScore *  
config.getWeightCompleteness());  
        return (int) Math.round(total);  
    }  
}
```

---

## 11. PdfReportService — Generate PDF (iText/OpenPDF)

```
@Service  
public class PdfReportService {  
  
    public String generate(int totalScore,  
ParsingResult parsingResult,  
                        AiAnalysisResult
```

```

aiResult, List<Issue> issues) {
    // 1. Buat Document baru (iText/OpenPDF)
    // 2. Tulis header (judul, total score
    besar)
    // 3. Tulis breakdown skor per gate
    // 4. Tulis list issue + severity
    // 5. Tulis rewrite suggestion per bullet
    // 6. Simpan ke storage (temp folder / S3),
    atau langsung return byte[] untuk stream
    // 7. Return URL/path untuk didownload
    ...
    return "/api/cv/report/" +
    UUID.randomUUID();
}
}

```

---

## 12. Config — Rules dari Luar Kode

```

@Configuration
@ConfigurationProperties(prefix = "ats.rules")
public class ScoringRulesConfig {
    private List<String> headingWhitelist;
    private List<String> fontWhitelist;
    private int minLength;
    private int keywordDensityMin;
    private int keywordDensityMax;
    private double weightParsing;
    private double weightKeyword;
    private double weightCompleteness;
    // getter & setter
}

```

```

# application.yml
ats:
  rules:

```

```
    heading-whitelist: [EXPERIENCE, EDUCATION,
SKILLS, SUMMARY, CERTIFICATIONS]
    font-whitelist: [Arial, Calibri, "Times New
Roman", Georgia]
    min-text-length: 200
    keyword-density-min: 2
    keyword-density-max: 3
    weight-parsing: 0.4
    weight-keyword: 0.4
    weight-completeness: 0.2
```

---

## 13. Exception Handling

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RateLimitExceededException.class)
    public ResponseEntity<ErrorResponse>
handleRateLimit(RateLimitExceededException e) {
        return ResponseEntity.status(429).body(new
ErrorResponse(e.getMessage()));
    }

    @ExceptionHandler(InvalidInputException.class)
    public ResponseEntity<ErrorResponse>
handleInvalidInput(InvalidInputException e) {
        return ResponseEntity.badRequest().body(new
ErrorResponse(e.getMessage()));
    }

    @ExceptionHandler(TextExtractionException.class)
    public ResponseEntity<ErrorResponse>
handleExtraction(TextExtractionException e) {
        return
```

```

        ResponseEntity.unprocessableEntity().body(new
        ErrorResponse(e.getMessage()));
    }

    @ExceptionHandler(AiResponseParsingException.class)
    public ResponseEntity<ErrorResponse>
    handleAiParsing(AiResponseParsingException e) {
        return ResponseEntity.status(502).body(new
        ErrorResponse("Gagal menganalisa, coba lagi"));
    }
}

public record ErrorResponse(String message) {}

```

## Ringkasan Urutan Eksekusi (sesuai

`CvAnalysisOrchestrator.process()` )

1. `RateLimitService.checkLimit()` → **429 kalau habis**
2. `FileValidationService.validate()` → **400 kalau invalid**
3. `TextExtractionService.extract()` → **422 kalau gagal baca**
4. `ParsingGateService.evaluate()` → **Gate 1, murni rule-based**
5. `AiAnalysisService.analyze()` → **Gate 2 & 3, panggil LLM, validasi JSON + retry**
6. `ScoringService.calculateTotal()` → **weighted formula dari config**
7. `PdfReportService.generate()` → **buat PDF report**
8. `repository.save()` → **kondisional, kalau user login**

## **9. Return `AnalysisResponse` ke frontend**

**Semua angka rules (whitelist font/heading, threshold density, bobot skor) hidup di `application.yml` , bukan hardcoded — biar bisa lo update tanpa recompile.**